

AgroSat

Monitoramento agrícola via dados de satélite

Global Solution 2026/1

4º Ano - Engenharia de Software - FIAP

Tema: Soluções para o desafio da Indústria Espacial | ODS 2 - Fome Zero

Integrantes do grupo

RM	Nome
552386	Guilherme Mazur
552413	Marina Pereira
86810	Bruno Lago
97833	Eduardo Sávio
551565	Gabriel Oliveira de Mattos
552322	Gustavo Vieira da Silva

Vídeo Pitch: _____

Sobre o projeto

O AgroSat é uma plataforma de monitoramento agrícola que leva dados antes restritos a agências espaciais (imagens de satélite, NDVI, clima) ao pequeno produtor rural. Integra índice de vegetação por satélite, sensores IoT de umidade do solo e uma engine de risco que gera alertas de seca e praga, acessíveis por um app mobile.

Este documento reúne, por parte do projeto, os entregáveis que não são código-fonte. O código completo (backend, mobile, scripts SQL e simulador IoT) está no .zip do projeto, com instruções no README.md.

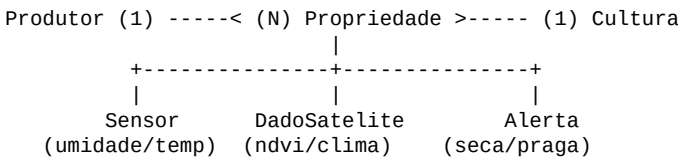
Conexão com a economia espacial: satélites de observação terrestre (Landsat, Sentinel, CBERS) como fonte primária de NDVI e clima, e GPS para georreferenciamento das propriedades — a mesma tecnologia usada por NASA e INPE, aplicada à agricultura de pequena escala.

1. Modelo de Banco de Dados

Entidades e relacionamentos

Entidade	Descrição	Relacionamentos
Produtor	Agricultor cadastrado	1:N com Propriedade
Propriedade	Talhão/fazenda georreferenciada	N:1 Produtor e Cultura; 1:N Sensor, DadoSatelite, Alerta
Cultura	Tipo de plantio (soja, milho)	1:N com Propriedade
Sensor	Leitura IoT (umidade do solo)	N:1 Propriedade
DadoSatelite	NDVI, precipitação, temperaturas	N:1 Propriedade
Alerta	Risco gerado (seca/praga)	N:1 Propriedade

Diagrama Entidade-Relacionamento



Entregáveis de código (no .zip): **banco/schema.sql** (CREATE TABLE com chaves primárias e estrangeiras, dialeto H2), **banco/seed.sql** (dados de demonstração) e **banco/consultas.sql**.

Consultas SQL de exemplo

#	Consulta	Para quê
1	Último NDVI por propriedade, classificado (saúdável/atenção/crítico)	Escala de cores do dashboard
2	Risco de seca: última umidade do solo + precipitação acumulada em 15 dias	Regra de seca (umidade<20% e chuva<10mm)
3	Alertas não lidos por produtor e severidade	Tela de alertas do app

2. Implementação dos Serviços (API)

API RESTful em Java 17 / Spring Boot 3.3.5, organizada em camadas Controller, Service e Repository, documentada com Swagger / OpenAPI 3 (disponível em /swagger-ui.html).

Método	Rota	Descrição
POST	/api/auth/login	Login do produtor (hash SHA-256)
POST	/api/produtores	Cadastrar produtor
GET	/api/propriedades	Listar propriedades
POST	/api/propriedades	Cadastrar propriedade
GET	/api/propriedades/{id}/ndvi	Último NDVI + status
GET	/api/propriedades/{id}/alertas	Alertas ativos
POST	/api/sensores/dados	Registrar leitura IoT
POST	/api/alertas/verificar	Rodar engine de risco

Documentação da API: Swagger UI em /swagger-ui.html (spec OpenAPI em /api-docs).

3. Plano de Testes

Casos de teste validando os serviços da API. Os 5 casos são executados pelo script **testes/run_tests.sh**, que compara o status HTTP e grava as evidências em **evidencias.log**.

#	Cenário	Entrada (resumo)	Esperado
TC01	Login válido	joao@agrosat.com.br / 123456	200
TC02	Login senha incorreta	joao@agrosat.com.br / errada	401
TC03	Cadastro email duplicado	email já existente no seed	400
TC04	Leitura IoT válida	UMIDADE_SOLO, 18.5, prop 1	201
TC05	Leitura prop. inexistente	UMIDADE_SOLO, 18.5, prop 999	404

Execução: rodar a aplicação, depois **bash testes/run_tests.sh**. O desafio exige no mínimo 5 casos e execução de ao menos 3; o script roda os 5 e o log serve de evidência (anexar print do resultado).

4. Front-end Mobile

App em React Native + TypeScript com 3 telas. Dados podem ser mockados (não exige consumo da API).

Tela	Conteúdo
Dashboard	Mapa de NDVI com escala de cores (verde/amarelo/vermelho), temperatura, umidade do solo e precipitação acumulada.
Alertas	Lista de alertas ativos com ícone de severidade, mensagem e ação de marcar como lido.
Sensores IoT	Sensores da propriedade com última leitura e barra de progresso vs. limites saudáveis.

Código no .zip em **mobile/** (telas em **mobile/src/screens**).

5. Segurança da Aplicação

Login com senha criptografada: POST `/api/auth/login` compara o hash SHA-256 da senha enviada com o hash armazenado; a senha nunca trafega nem é comparada em texto puro. Resposta 401 genérica para email inexistente ou senha errada (evita enumeração de usuários).

Prática	Implementação
Criptografia de senha	SHA-256 (MessageDigest); armazena só o digest
Validação de entrada	Bean Validation (<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@NotNull</code>) com <code>@Valid</code>
Proteção contra SQL Injection	Spring Data JPA com consultas parametrizadas (sem concatenação)
CORS controlado	WebMvcConfigurer liberando apenas <code>/api/**</code>
Erros padronizados	<code>@RestControllerAdvice</code> com ProblemDetail (RFC 9457)

Evolução projetada: autenticação JWT stateless (login emite token, filtro `OncePerRequestFilter` valida, `SecurityFilterChain` libera rotas públicas) e migração do hash para BCrypt. Detalhada em `docs/seguranca.md`.

6. Simulação de IoT

Simulador em Python (**iot/simulador_sensor.py**) que envia leituras para POST /api/sensores/dados.

Aspecto	Descrição
Qual sensor	Umidade do solo (TipoSensor.UMIDADE_SOLO), simulando sensor capacitivo de campo
O que mede	Percentual de umidade do solo, valores entre 15% e 35% com variação gradual (passeio aleatório)
Como influencia o sistema	Cada leitura alimenta a regra de seca: umidade < 20% e precipitação acumulada em 15 dias < 10mm geram um Alerta de SECA, exibido no painel e na tela de alertas. O dado do dispositivo vira decisão automática para o produtor.

Tecnologias

Backend: Java 17, Spring Boot 3.3.5, Spring Data JPA, Hibernate. Banco: H2 (dev) / PostgreSQL (prod).
Documentação: Swagger/OpenAPI 3. Mobile: React Native + TypeScript. IoT: Python. Testes: script curl + (opcional) Spring Boot Test.